JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface InvocationHandler

All Known Implementing Classes:

CompositeDataInvocationHandler, EventHandler, MBeanServerInvocationHandler, RemoteObjectInvocationHandler

---

public interface **InvocationHandler**

InvocationHandler is the interface implemented by the *invocation handler* of a proxy instance.

Each proxy instance has an associated invocation handler. When a method is invoked on a proxy instance, the method invocation is encoded and dispatched to the invoke method of its invocation handler.

Since:

1.3

See Also:

Proxy

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
Object	**invoke**(Object proxy, Method method, Object[] args)	Processes a method invocation on a proxy instance and returns the result.
static Object	**invokeDefault**(Object proxy, Method method, Object... args)	Invokes the specified default method on the given proxy instance with the given parameters.

Method Details

invoke

```
Object invoke(Object proxy,
              Method method,
              Object[] args)
       throws Throwable
```

Processes a method invocation on a proxy instance and returns the result. This method will be invoked on an invocation handler when a method is invoked on a proxy instance that it is associated with.

Parameters:

`proxy` - the proxy instance that the method was invoked on

`method` - the `Method` instance corresponding to the method invoked on the proxy instance; the declaring class of the `Method` object may be a proxy interface, one of their superinterfaces, or the `Object` class

`args` - an array of objects containing the values of the arguments passed in the method invocation on the proxy instance, or `null` if the invoked method takes no arguments. Arguments of primitive types are wrapped in instances of the appropriate primitive wrapper class, such as `java.lang.Integer` or `java.lang.Boolean`.

Returns:

the value to return from the method invocation on the proxy instance. If the declared return type of the invoked method is a primitive type, then the value returned by this method must be an instance of the corresponding primitive wrapper class; otherwise, it must be a type assignable to the declared return type. If the value returned by this method is `null` and the invoked method's return type is primitive, then a `NullPointerException` will be thrown by the method invocation on the proxy instance. If the value returned by this method is otherwise not compatible with the invoked method's declared return type as described above, a `ClassCastException` will be thrown by the method invocation on the proxy instance.

Throws:

`Throwable` - the exception to throw from the method invocation on the proxy instance. The exception's type must be assignable either to any of the exception types declared in the `throws` clause of the invoked method or to the unchecked exception types `java.lang.RuntimeException` or `java.lang.Error`. If a checked exception is thrown by this method that is not assignable to any of the exception types declared in the `throws` clause of the invoked method, then an `UndeclaredThrowableException` containing the exception that was thrown by this method will be thrown by the method invocation on the proxy instance.

See Also:

`UndeclaredThrowableException`

invokeDefault

```
static Object invokeDefault(Object proxy,
                            Method method,
                            Object... args)
                     throws Throwable
```

Invokes the specified default method on the given `proxy` instance with the given parameters. The given `method` must be a default method declared in a proxy interface of the `proxy`'s class or inherited from its superinterface directly or indirectly.

Invoking this method behaves as if `invokespecial` instruction executed from the proxy class, targeting the default method in a proxy interface. This is equivalent to the invocation: `X.super.m(A* a)` where `X` is a proxy interface and the call to `X.super::m(A*)` is resolved to the given `method`.

Examples: interface `A` and `B` both declare a default implementation of method `m`. Interface `C` extends `A` and inherits the default method `m` from its superinterface `A`.

```
interface A {
    default T m(A a) { return t1; }
}
interface B {
    default T m(A a) { return t2; }
}
interface C extends A {}
```

The following creates a proxy instance that implements `A` and invokes the default method `A::m`.

```
Object proxy = Proxy.newProxyInstance(loader, new Class<?>[] { A.class },
        (o, m, params) -> {
            if (m.isDefault()) {
                // if it's a default method, invoke it
                return InvocationHandler.invokeDefault(o, m, params);
            }
        });
```

If a proxy instance implements both `A` and `B`, both of which provides the default implementation of method `m`, the invocation handler can dispatch the method invocation to `A::m` or `B::m` via the `invokeDefault` method. For example, the following code delegates the method invocation to `B::m`.

```
Object proxy = Proxy.newProxyInstance(loader, new Class<?>[] { A.class, B.cl
        (o, m, params) -> {
            if (m.getName().equals("m")) {
                // invoke B::m instead of A::m
                Method bMethod = B.class.getMethod(m.getName(), m.getParamet
                return InvocationHandler.invokeDefault(o, bMethod, params);
            }
        });
```

If a proxy instance implements `C` that inherits the default method `m` from its superinterface `A`, then the interface method invocation on `"m"` is dispatched to the invocation handler's `invoke` method with the `Method` object argument representing the default method `A::m`.

```
Object proxy = Proxy.newProxyInstance(loader, new Class<?>[] { C.class },
       (o, m, params) -> {
            if (m.isDefault()) {
                // behaves as if calling C.super.m(params)
                return InvocationHandler.invokeDefault(o, m, params);
            }
       });
```

The invocation of method `"m"` on this `proxy` will behave as if `C.super::m` is called and that is resolved to invoking `A::m`.

Adding a default method, or changing a method from abstract to default may cause an exception if an existing code attempts to call `invokeDefault` to invoke a default method. For example, if `C` is modified to implement a default method `m`:

```
interface C extends A {
    default T m(A a) { return t3; }
}
```

The code above that creates proxy instance `proxy` with the modified `C` will run with no exception and it will result in calling `C::m` instead of `A::m`.

The following is another example that creates a proxy instance of `C` and the invocation handler calls the `invokeDefault` method to invoke `A::m`:

```
C c = (C) Proxy.newProxyInstance(loader, new Class<?>[] { C.class },
        (o, m, params) -> {
            if (m.getName().equals("m")) {
```

```
                // IllegalArgumentException thrown as {@code A::m} is not a
                // inherited from its proxy interface C
                Method aMethod = A.class.getMethod(m.getName(), m.getParamet
                return InvocationHandler.invokeDefault(o, aMethod params);
            }
        });
c.m(...);
```

The above code runs successfully with the old version of `C` and `A::m` is invoked. When running with the new version of `C`,
the above code will fail with `IllegalArgumentException` because `C` overrides the implementation of the same method and `A::m` is not accessible by a proxy instance.

API Note:

The `proxy` parameter is of type `Object` rather than `Proxy` to make it easy for `InvocationHandler::invoke` implementation to call directly without the need of casting.

Parameters:

`proxy` - the `Proxy` instance on which the default method to be invoked

`method` - the `Method` instance corresponding to a default method declared in a proxy interface of the proxy class or inherited from its superinterface directly or indirectly

`args` - the parameters used for the method invocation; can be `null` if the number of formal parameters required by the method is zero.

Returns:

the value returned from the method invocation

Throws:

`IllegalArgumentException` - if any of the following conditions is `true`:

- `proxy` is not a proxy instance; or
- the given `method` is not a default method declared in a proxy interface of the proxy class and not inherited from any of its superinterfaces; or
- the given `method` is overridden directly or indirectly by the proxy interfaces and the method reference to the named method never resolves to the given `method`; or
- the length of the given `args` array does not match the number of parameters of the method to be invoked; or
- any of the `args` elements fails the unboxing conversion if the corresponding method parameter type is a primitive type; or if, after possible unboxing, any of the `args` elements cannot be assigned to the corresponding method parameter type.

`IllegalAccessException` - if the declaring class of the specified default method is inaccessible to the caller class

`NullPointerException` - if `proxy` or `method` is `null`

`Throwable` - anything thrown by the default method

See *Java Virtual Machine Specification*:

5.4.3 Resolution

Since:

16

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.